

Ex 6

Simulation of a Cloud Scenario using CloudSim and Implementation of a Priority Scheduling Algorithm

Aim:

To simulate a simple cloud scenario using CloudSim and implement a priority-based scheduling algorithm for executing cloud tasks.

Prerequisites

- Java JDK
- Eclipse IDE
- CloudSim library

Procedure

Step 1: Create the Java Project

Open Eclipse and create a Java project named "cloudsimscheduling". Add the CloudSim library to the project.

Step 2: Create the Program

Create a Java class named "Experiment6" inside the src folder.

Step 3: Create Cloud Tasks

Create four cloud tasks (Cloudlet 1, 2, 3 and 4) and assign priorities 2, 5, 1 and 4 respectively.

Step 4: Apply Priority Scheduling

Compare the priorities of the tasks and arrange them from highest to lowest priority. The task with the highest priority is scheduled first.

Step 5: Execute the Simulation

Run the program in Eclipse and display the scheduling order in the Console.

Program

```
package org.cloudbus.cloudsim.examples;

import org.cloudbus.cloudsim.*;
import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.provisioners.BwProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.PeProvisionerSimple;
import org.cloudbus.cloudsim.provisioners.RamProvisionerSimple;
```

```

import java.text.DecimalFormat;
import java.util.ArrayList;
import java.util.Calendar;
import java.util.LinkedList;
import java.util.List;

public class SJFSimulation {

    public static void main(String[] args) {

        System.out.println("Starting SJF CloudSim simulation...");

        try {
            // 1. Initialize CloudSim
            CloudSim.init(
                1,
                Calendar.getInstance(),
                false
            );

            // 2. Create datacenter
            createDatacenter("Datacenter_0");

            // 3. Create the custom SJF broker
            SJFDatacenterBroker broker =
                new SJFDatacenterBroker("SJF_Broker");

            int brokerId = broker.getId();

            // 4. Create one virtual machine
            List<Vm> vmList = new ArrayList<>();

            Vm vm = new Vm(
                0, // VM ID
                brokerId,
                1000, // MIPS
                1, // Number of CPU cores
                512, // RAM in MB
                1000, // Bandwidth
                10000, // Storage
                "Xen",
                new CloudletSchedulerSpaceShared()
            );

            vmList.add(vm);
            broker.submitGuestList(vmList);

            // 5. Create cloudlets with different lengths
            List<Cloudlet> cloudletList = new ArrayList<>();
            UtilizationModel utilization = new UtilizationModelFull();

            // Longest job
            Cloudlet cloudlet0 = new Cloudlet(
                0, 40000, 1, 300, 300,
                utilization, utilization, utilization
            );
            cloudlet0.setUserId(brokerId);

            // Shortest job
            Cloudlet cloudlet1 = new Cloudlet(
                1, 10000, 1, 300, 300,
                utilization, utilization, utilization
            );

```

```

        cloudlet1.setUserId(brokerId);

        // Medium job
        Cloudlet cloudlet2 = new Cloudlet(
            2, 20000, 1, 300, 300,
            utilization, utilization, utilization
        );
        cloudlet2.setUserId(brokerId);

        // Intentionally submit them in unsorted order
        cloudletList.add(cloudlet0);
        cloudletList.add(cloudlet1);
        cloudletList.add(cloudlet2);

        broker.submitCloudletList(cloudletList);

        // 6. Run simulation
        CloudSim.startSimulation();

        List<Cloudlet> completedCloudlets =
            broker.getCloudletReceivedList();

        CloudSim.stopSimulation();

        // 7. Display results
        printCloudletList(completedCloudlets);

        System.out.println("\nSJF simulation completed.");
    } catch (Exception exception) {
        exception.printStackTrace();
    }
}

private static Datacenter createDatacenter(String name) {

    List<Pe> peList = new ArrayList<>();
    peList.add(new Pe(
        0,
        new PeProvisionerSimple(1000)
    ));

    List<Host> hostList = new ArrayList<>();

    Host host = new Host(
        0,
        new RamProvisionerSimple(2048),
        new BwProvisionerSimple(10000),
        1000000,
        peList,
        new VmSchedulerTimeShared(peList)
    );

    hostList.add(host);

    DatacenterCharacteristics characteristics =
        new DatacenterCharacteristics(
            "x86",
            "Linux",
            "Xen",
            hostList,
            10.0,
            3.0,
            0.05,

```

```

        0.001,
        0.0
    );

    try {
        return new Datacenter(
            name,
            characteristics,
            new VmAllocationPolicySimple(hostList),
            new LinkedList<>(),
            0
        );
    } catch (Exception exception) {
        throw new RuntimeException(exception);
    }
}

private static void printCloudletList(List<Cloudlet> cloudlets) {

    DecimalFormat format = new DecimalFormat("0.##");

    System.out.println(
        "\n===== SIMULATION RESULTS ====="
    );

    System.out.printf(
        "%-12s %-10s %-8s %-12s %-12s %-12s\n",
        "Cloudlet ID",
        "Status",
        "VM ID",
        "CPU Time",
        "Start Time",
        "Finish Time"
    );

    for (Cloudlet cloudlet : cloudlets) {

        if (cloudlet.getCloudletStatus() == Cloudlet.SUCCESS) {
            System.out.printf(
                "%-12d %-10s %-8d %-12s %-12s %-12s\n",
                cloudlet.getCloudletId(),
                "SUCCESS",
                cloudlet.getVmId(),
                format.format(cloudlet.getActualCPUTime()),
                format.format(cloudlet.getExecStartTime()),
                format.format(cloudlet.getFinishTime())
            );
        }
    }
}
}

```

Output:

```
Sep 7 11:03
vboxuser@ubuntu: ~/cloudsim/modules/cloudsim-examples
~/cloudsim/modules/cloudsim-examples

vboxuser@ubuntu: ~/minio — minio server /home/vboxuser/minio-data --console-address: vboxuser@ubuntu: ~/cloudsim/modules/cloudsim-examples

Cloudlet 1 - Length: 10000 MI
Cloudlet 2 - Length: 20000 MI
Cloudlet 0 - Length: 40000 MI
0.01: SJF_Broker: Sending Cloudlet #1 to Vm #0
0.01: SJF_Broker: Sending Cloudlet #2 to Vm #0
0.01: SJF_Broker: Sending Cloudlet #0 to Vm #0
10.01: SJF_Broker: Cloudlet #1 return received
10.01: SJF_Broker: The number of finished Cloudlets is:1
30.009999999999998: SJF_Broker: Cloudlet #2 return received
30.009999999999998: SJF_Broker: The number of finished Cloudlets is:2
70.009999999999999: SJF_Broker: Cloudlet #0 return received
70.009999999999999: SJF_Broker: The number of finished Cloudlets is:3
70.009999999999999: SJF_Broker: All Cloudlets executed. Finishing...
70.009999999999999: SJF_Broker: Destroying Vm #0
70.009999999999999: SJF_Broker is shutting down...
70.009999999999999: Simulation: No more future events
70.009999999999999: CloudInformationService is shutting down...
70.009999999999999: CloudInformationService: Notify all CloudSim entities for shutting down.
70.009999999999999: Datacenter_0 is shutting down...
70.009999999999999: SJF_Broker is shutting down...
Simulation completed.
Simulation completed.

===== SIMULATION RESULTS =====
Cloudlet ID  Status    VM ID   CPU Time   Start Time  Finish Time
1            SUCCESS    0       10         0.01        10.01
2            SUCCESS    0       20         10.01       30.01
0            SUCCESS    0       40         30.01       70.01

SJF simulation completed.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.874 s
[INFO] Finished at: 2026-09-07T11:02:39Z
[INFO] -----
vboxuser@ubuntu:~/cloudsim/modules/cloudsim-examples$
```

Result:

Thus, the cloud scheduling scenario was successfully simulated and the priority-based scheduling algorithm was implemented. The tasks were arranged and scheduled according to their priority.